```
php artisan make:model Car --controller --resource --requests
php artisan make:model Car -crR
# Generate a model and a CarPolicy class
php artisan make:model Car --policy
# Generate a model and a migration, factory, seeder, and
controller
php artisan make:model Car -mfsc
# Shortcut to generate a model, migration, factory, seeder,
policy, controller, and form requests
php artisan make:model Car --all
# Generate a pivot model
php artisan make:model Member --pivot
php artisan make:model Member -p
```

Database Migrations

Database migrations play a crucial role in managing version control for databases, allowing your team to formulate and distribute the schema definition for the application's database. If you've ever encountered the challenge of instructing a team member to manually integrate a new column into their local database schema after pulling your changes from source control, you've likely grappled with the problem that database migrations aim to solve.

The Laravel Schema facade offers a versatile solution for creating and manipulating tables in a manner that is independent of the underlying database system. This facade is commonly employed in migrations to facilitate the creation and modification of database tables and columns across all of Laravel's supported database systems.

To create a database migration, use the **`make:migration`** Artisan command. The newly created migration will be saved in your **`database/migrations`** directory. Each migration filename has a date, which helps Laravel to identify the migrations' order:

```
php artisan make:migration create_cars_table
```

Laravel endeavors to predict the table name and determine whether a migration will generate a new table by analyzing the migration's name. If Laravel can deduce the table name from the migration name, it automatically populates the created migration file with the identified table. However, if the